In the OpenAI Gym API, environments with discrete action spaces all have actions represented as integers from 0 to the total number of actions for the particular environment, so in CartPole the possible actions are 0 and 1, which denote a push to the left or to the right. The state is represented as a vector of length 4 that indicates the cart position, cart velocity, pole angle, and pole velocity. We receive a reward of +1 for every step the pole has not fallen over, which happens when the pole angle is more than 12° from the center or when the cart position is outside the window. Hence, the goal of CartPole is to maximize the length of the episode, since each step returns a positive +1 reward. More information can be found on the OpenAI Gym GitHub page (https://github.com/openai/gym/wiki/CartPole-v0). Note that not every subsequent problem has a nice specification page like CartPole does, but we will define the scope of the problem beforehand in all subsequent chapters.

4.3.2. The OpenAI Gym API

The OpenAI Gym has been built to be incredibly easy to use, and there are less than half a dozen methods that you'll routinely use. You already saw one in listing 4.1 where we listed all the available environments. Another important method is creating an environment.

Listing 4.2. Creating an environment in OpenAI Gym

```
import gym
env = gym.make('CartPole-v0')
```

From now on, we will be interacting solely with this `env` variable. We need a way to observe the current state of the environment and then to interact with it. There are only two methods you need to do this.

Listing 4.3. Taking an action in CartPole

```
state1 = env.reset()
action = env.action_space.sample()
state, reward, done, info = env.step(action)
```

The `reset` method initializes the environment and returns the first state. For this example, we used the `sample` method of the